

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KHOA HỌC MÁY TÍNH



ĐỒ ÁN MÔN HỌC

CS231 - NHẬP MÔN THỊ GIÁC MÁY TÍNH

ĐỀ TÀI: PHÂN LOẠI CHI TIẾT LOÀI CHIM (FINE-GRAINED
CLASSIFICATION) TRÊN TẬP DỮ LIỆU BIRDS-525

Nhóm sinh viên:

Nguyễn Duy Hoàng Anh – 24520101

Nguyễn Ngọc Anh Tuấn – 24520034

Trần Nguyễn Lâm Huy – 24520704

Nguyễn Bá Toàn – 24521796

Mã lớp:

CS231.Q23

TP.HCM, ngày 27 tháng 04 năm 2026

Mục lục

1	Lý do chọn đề tài	1
2	Mô tả bài toán	1
3	Phương pháp xử lý dữ liệu	2
3.1	Chia tập dữ liệu	2
3.2	Tăng cường dữ liệu tập train (Data Augmentation)	4
4	Phương pháp trích xuất đặc trưng	6
4.1	Phương pháp VGG16	6
4.1.1	Tổng quan về VGG16	6
4.1.2	Trích xuất đặc trưng từ VGG16 pretrained trên ImageNet	6
4.1.3	Phân loại bằng Random Forest	8
4.2	Phương pháp Vision Transformer (ViT)	10
4.2.1	Tổng quan về ViT	10
4.2.2	Kiến trúc Mô hình ViT	10
4.2.3	Mã nguồn ViT[1]	14
4.2.4	Lý do lựa chọn ViT-B/16 để rút trích đặc trưng	15
4.3	Phương pháp 3: EfficientNetB0	16
4.3.1	Tổng quan về EfficientNetB0	16
4.3.2	Trích xuất đặc trưng	18
4.3.3	Cấu hình Feature Extractor	19
4.4	Phương pháp CLIP	19
4.4.1	Tổng quan về CLIP (Contrastive Language-Image Pre-training)	19
4.4.2	Huấn luyện mô hình	22
5	Mô hình phân loại	23

6	Thực nghiệm và kết quả	25
6.1	Đánh giá bộ dữ liệu	25
6.2	Thực nghiệm	26
6.3	Kết quả thực nghiệm	27
6.4	Nhận xét kết quả	28
6.4.1	So sánh tổng thể 4 phương pháp	28
6.4.2	Ảnh hưởng của tăng cường dữ liệu (Data Augmentation) . . .	29
7	Đánh giá, kết luận và mở rộng:	30
7.1	Đánh giá	30
7.1.1	Ưu điểm	30
7.1.2	Hạn chế	31
7.2	Kết luận	31
7.3	Mở rộng	32
	TÀI LIỆU THAM KHẢO	33

Danh sách hình ảnh

1	Ảnh minh họa	2
2	Thống kê số lượng ảnh	4
3	Thống kê số lượng ảnh trước khi tăng cường dữ liệu (Tập Train)	5
4	Thống kê số lượng ảnh sau khi tăng cường dữ liệu (Tập Train)	5
5	Kiến trúc mạng VGG16	6
6	Quy trình phân loại bằng Random Forest	8
7	Kiến trúc của Vision Transformer	10
8	Minh họa quá trình Linear Projection của các Patch trong ViT	11
9	Kiến trúc Transformer Encoder	12
10	Cấu trúc Self Attention	13
11	Cấu trúc Multi Head Attention	14
12	Classification Head	14
13	So sánh các phương pháp mở rộng mạng (Scaling) và cơ chế Compound Scaling của EfficientNet.	16
14	Sơ đồ pipeline của mô hình CLIP	20
15	Kiến trúc của mô hình CLIP.	20
16	Cơ chế hoạt động của CLIP khi huấn luyện bằng contrastive learning.	21
17	Công thức tính CLIP Contrastive Loss	21
18	Sơ đồ các giai đoạn huấn luyện mô hình	22
19	Các ví dụ phân loại đúng của ViT	27
20	Phân loại sai của phương pháp ViT	28
21	Phân loại sai của phương pháp ViT	28

Danh sách bảng

1	Vai trò của các tập dữ liệu	3
2	Cấu hình bộ trích xuất đặc trưng VGG16	8
3	Không gian tìm kiếm siêu tham số Random Forest bằng Grid Search	9
4	Bộ siêu tham số tốt nhất của Random Forest sau Grid Search	9
5	Kiến trúc của EfficientNet B0	17
6	Cấu hình bộ trích xuất đặc trưng EfficientNetB0	19
7	Bảng so sánh kết quả sử dụng tài nguyên trên bốn phương pháp giải quyết	27
8	So sánh kết quả với và không sử dụng Data Augmentation	27

Thông tin thành viên

STT	Họ và tên	MSSV	Công việc	Mức độ hoàn thành
1	Nguyễn Bá Toàn	24521796	Thu thập dữ liệu, rút trích đặc trưng bằng Vision Transformer và sử dụng mô hình phân loại RandomForest. Hoàn thành báo cáo, slide và thực hiện demo trên ViT-B/16	100%
2	Nguyễn Ngọc Anh Tuấn	24520034	Chia tập dữ liệu, rút trích đặc trưng EfficientNetB0 và sử dụng mô hình phân loại RandomForest. Hoàn thành báo cáo, slide và thực hiện demo trên EfficientNetB0	100%
3	Trần Nguyễn Lâm Huy	24520704	Tăng cường dữ liệu, rút trích đặc trưng VGG16 và sử dụng mô hình phân loại RandomForest. Hoàn thành báo cáo, slide và thực hiện demo trên VGG16	100%
4	Nguyễn Duy Hoàng Anh	24520101	Thu thập và tổng hợp dữ liệu, rút trích đặc trưng bằng CLIP, tìm tham số bằng GridSearchCV, sử dụng mô hình phân loại RandomForest. Hoàn thành báo cáo, slide và thực hiện demo trên CLIP ViT-B/32	100%

1 Lý do chọn đề tài

Bài toán phân loại chi tiết (fine-grained classification) tập trung vào việc phân biệt các đối tượng nằm trong cùng một nhóm nhưng chỉ khác nhau bởi những đặc điểm rất nhỏ về hình dạng hoặc ngoại hình. Do đó, mô hình không chỉ dựa vào các đặc trưng tổng quát mà cần khai thác các đặc điểm tinh vi và chi tiết hơn. Điều này đòi hỏi khả năng học sâu các đặc trưng nhằm phân biệt hiệu quả giữa các lớp có mức độ tương đồng cao.

Nhóm chọn thực hiện hệ thống nhận diện loài chim, là một hệ thống thuộc bài toán phân loại chi tiết (fine-grained classification), nhằm phân biệt các loài chim với số lượng đa dạng và thường có sự khác biệt rất nhỏ về đặc điểm ngoại hình, dẫn đến dễ bị nhầm lẫn. Hệ thống nhận diện loài chim có nhiều ứng dụng thực tiễn quan trọng:

- **Giám sát môi trường và bảo tồn:** Hệ thống cho phép theo dõi đa dạng sinh học, phát hiện các loài quý hiếm và đánh giá tình trạng hệ sinh thái thông qua sự biến động của quần thể chim.
- **Giáo dục và Du lịch sinh thái:** Hệ thống này có thể tích hợp vào các bảng thông tin tương tác tại các vườn quốc gia hoặc khu bảo tồn, giúp khách du lịch tìm hiểu về hệ sinh thái địa phương một cách sinh động mà không cần hướng dẫn viên đi kèm mọi lúc
- **Sử dụng trong các trung tâm cứu hộ động vật:** Hệ thống giúp xác định loài chim để xây dựng phương án chăm sóc phù hợp, đồng thời hỗ trợ quản lý và theo dõi quá trình phục hồi một cách hiệu quả.

2 Mô tả bài toán

Tên bài toán: Phân loại chi tiết (Fine-grained Classification) loài chim sử dụng tập dữ liệu Birds-525

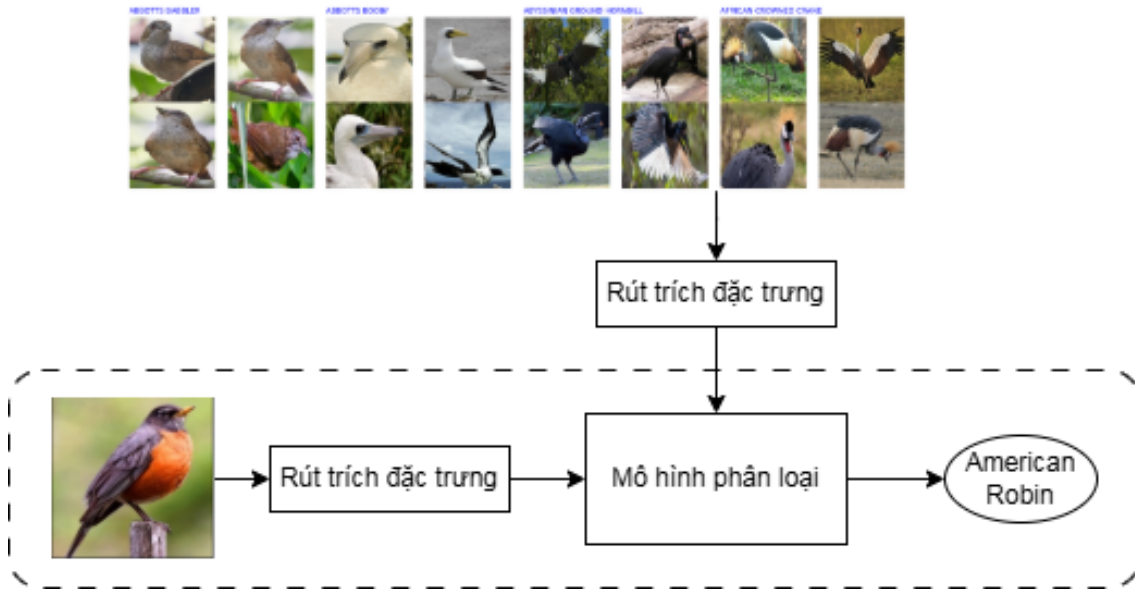
Phạm vi thực hiện: Trong khuôn khổ đề án, thay vì thực hiện trên toàn bộ 150 loài, nhóm tập trung nghiên cứu và thực nghiệm trên 150 loài tiêu biểu nhất (lọc theo số lượng dữ liệu lớn nhất). Việc giới hạn này giúp đảm bảo tính hội tụ của mô hình và cho phép phân tích sâu hơn về các đặc trưng thị giác tinh vi giữa các loài có độ tương đồng cao. Đặc biệt, về tính thực nghiệm, quy mô này cho phép nhóm thực hiện nhiều lượt huấn luyện để tìm ra kiến trúc mạng tối ưu, thay vì chỉ thực hiện được số lần hạn chế trên toàn bộ tập dữ liệu do các giới hạn về thời gian và

tài nguyên phần cứng.

Input:

1. Tập dữ liệu gồm N ảnh đã được gán nhãn: $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$
 - Tập các nhãn $\mathcal{D}$ là: $\mathcal{L} = \bigcup_{i=1}^N y_i$, với $\mathcal{L}$ là tập tên các loài chim khác nhau.
(Ví dụ: ABBOTTS_BABBLER, BARN_OWL, CHIPPING_SPARROW)
 - Mỗi phần tử của $\mathcal{D}$:
 - $\mathbf{x}_i$ là ảnh số $\mathcal{I}$ với định dạng màu RGB của một con chim, $\mathbf{x}_i \in \mathbb{R}^{h \times w \times 3}$.
 - y_i là giá trị nhãn (label) tên của loài chim tương ứng được gán cho $\mathbf{x}_i$.
2. Ảnh $\mathbf{x} \in \mathcal{I}$

Output: $\hat{y} \in \mathcal{L}$: Giá trị nhãn dự đoán của ảnh $\mathbf{x}$.



Hình 1: Ảnh minh họa

3 Phương pháp xử lý dữ liệu

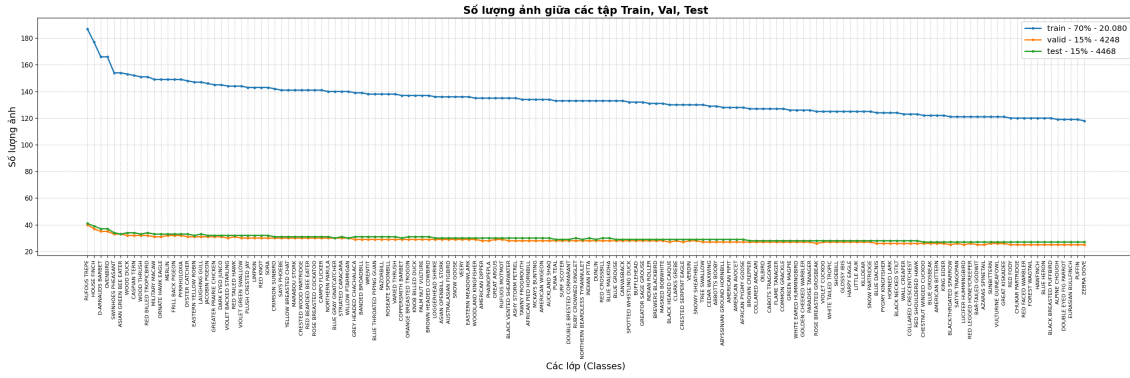
3.1 Chia tập dữ liệu

- Để đảm bảo quá trình huấn luyện diễn ra công bằng và hạn chế hiện tượng lệch phân phối dữ liệu, bộ dữ liệu đã được chia sẵn thành ba phần: tập huấn luyện (train), tập kiểm định (valid) và tập kiểm tra (test) với tỷ lệ tương ứng là 70% – 15% – 15%

Bảng 1: Vai trò của các tập dữ liệu

Loại tập	Mục đích chính	Kết quả đạt được	Chỉ số đánh giá	Lỗi thường gặp
Train	Huấn luyện mô hình, học các tham số	Mô hình học được các đặc trưng từ dữ liệu đầu vào	Training Loss, Training Accuracy	Dữ liệu không đủ dẫn đến underfitting hoặc không đại diện
Valid	Tinh chỉnh siêu tham số, chọn mô hình, phát hiện overfitting	Xác định siêu tham số tối ưu, hỗ trợ early stopping	Validation Loss, Validation Accuracy, F1-score	Over-tuning, rò rỉ dữ liệu từ tập train
Test	Đánh giá cuối cùng, kiểm tra khả năng tổng quát hóa	Ước lượng hiệu năng thực tế của mô hình	Test Accuracy, Precision, Recall, mAP, Dice Score	Peeking test set, rò rỉ dữ liệu, không đại diện thực tế

- Toàn bộ dữ liệu đã nhóm tải lên dưới dạng thư mục dataset, bao gồm ba thư mục con: train, val và test. Trong mỗi thư mục con, các ảnh được phân loại theo từng lớp riêng biệt.
- Mỗi ảnh đã được gán nhãn sẵn dựa trên cấu trúc thư mục, do đó việc nạp dữ liệu có thể thực hiện trực tiếp thông qua ImageFolder mà không cần xử lý thêm.
- Cách tổ chức này giúp duy trì sự đồng đều trong phân phối lớp giữa các tập dữ liệu, từ đó hỗ trợ mô hình học được các đặc trưng tổng quát tốt hơn và đảm bảo kết quả đánh giá phản ánh chính xác hiệu năng thực tế.



Hình 2: Thống kê số lượng ảnh

3.2 Tăng cường dữ liệu tập train (Data Augmentation)

Data augmentation là kỹ thuật tăng cường dữ liệu huấn luyện bằng cách tạo ra các biến thể mới của ảnh gốc. Các biến thể này được sinh ra thông qua các cách biến đổi như xoay, lật, thay đổi độ sáng, phóng to, thu nhỏ hay thêm nhiễu trên ảnh gốc. Kỹ thuật này giúp mô hình học được các đặc trưng bất biến trước sự đa dạng của góc độ ảnh và điều kiện ánh sáng, đồng thời còn hạn chế được tình trạng overfitting, đặc biệt là khi số lượng ảnh trong một số lớp loài chim còn chưa nhiều (khoảng 150 ảnh mỗi lớp).

1. Bộ dữ liệu có số lượng ảnh còn chênh lệch giữa các lớp, với lớp nhiều nhất gồm 187 ảnh, lớp ít nhất gồm 118 ảnh. Do đó, nhóm sử dụng augmentation giúp cân bằng lại số lượng mẫu giữa các loài chim, tăng tính đại diện cho từng lớp.
2. Các ảnh trong bộ dữ liệu có nhiều biến đổi về góc chụp, tư thế, nền và ánh sáng, vì vậy mô hình cần có đủ dữ liệu để nhận biết các đặc trưng không thay đổi trước những yếu tố này. Áp dụng data augmentation giúp nhóm mở rộng tập huấn luyện và cải thiện độ tổng quát của mô hình, nhằm nâng cao độ chính xác khi dự đoán trên ảnh thực tế mới.

```

1  #flip horizontal
2  if random.random() < 0.5:
3      img = img.transpose(Image.Transpose.FLIP_LEFT_RIGHT)
4  #random rotation in range [-8, 8]
5  angle = random.uniform(-8, 8)
6  img = img.rotate(
7      angle,
8      resample=Image.Resampling.BICUBIC,
9      fillcolor=(0, 0, 0))

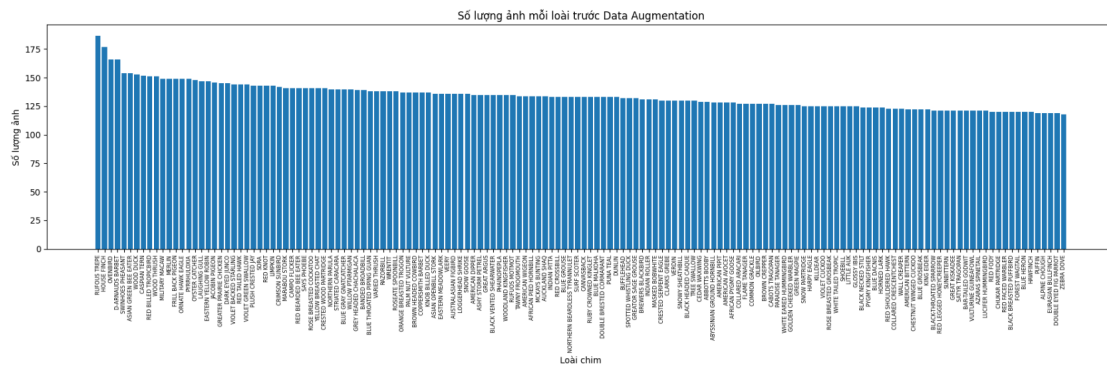
```

```

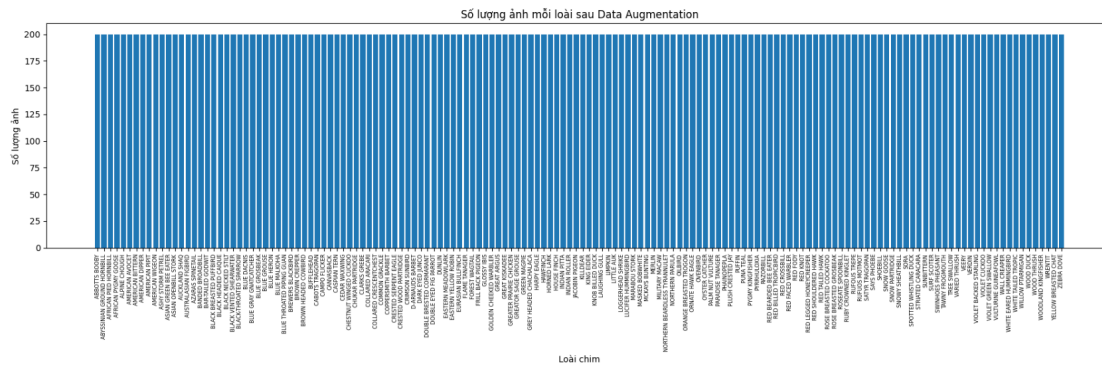
10 #adjust brightness
11 brightness_factor = random.uniform(0.85, 1.15)
12 img = ImageEnhance.Brightness(img).enhance(brightness_factor)

```

Trước khi tăng cường dữ liệu, ta thấy được số lượng ảnh giữa các lớp loài chim ở tập train có sự chênh lệch, với số mẫu mỗi lớp dao động khoảng từ 100 đến trên 180 ảnh. Nhóm quyết định thực hiện tăng cường dữ liệu sao cho tất cả các lớp đều có số lượng mẫu bằng nhau và bằng 200. Việc tăng cường đến 200 ảnh giúp cho những loài chim đã có nhiều ảnh cũng được bổ sung thêm các góc xoay và ánh sáng. Số lượng ảnh sau khi tăng cường là 30000 ảnh, tăng 9920 ảnh so với bộ dữ liệu train ban đầu (20,080 ảnh).



Hình 3: Thống kê số lượng ảnh trước khi tăng cường dữ liệu (Tập Train)



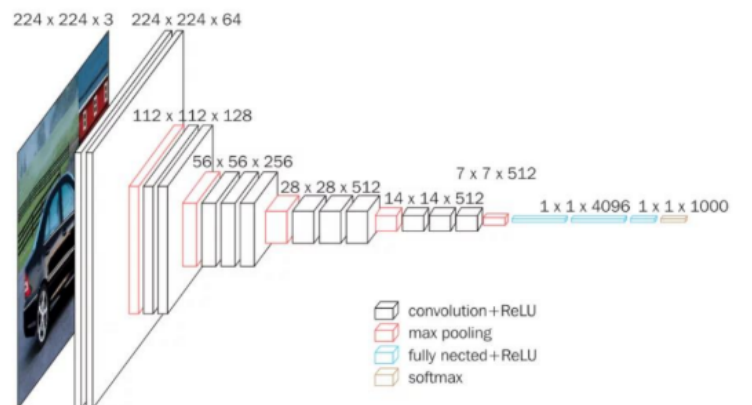
Hình 4: Thống kê số lượng ảnh sau khi tăng cường dữ liệu (Tập Train)

4 Phương pháp trích xuất đặc trưng

4.1 Phương pháp VGG16

4.1.1 Tổng quan về VGG16

- **Giới thiệu:** VGG16 là kiến trúc mạng nơ-ron tích chập được đề xuất bởi Karen Simonyan và Andrew Zisserman thuộc nhóm Visual Geometry Group, Đại học Oxford [2]. Mô hình này được huấn luyện trên tập ImageNet và thường được sử dụng trong các bài toán phân loại ảnh, nhận dạng đối tượng và trích xuất đặc trưng ảnh.
- **Đặc điểm chính:** VGG16 gồm 13 lớp tích chập và 3 lớp fully connected trong kiến trúc gốc [2]. Nhóm không sử dụng phần fully connected gốc của VGG16 mà chỉ sử dụng phần backbone tích chập với trọng số pretrained trên ImageNet để trích xuất đặc trưng ảnh.
- **Lý do lựa chọn:** VGG16 có kiến trúc đơn giản, dễ triển khai và có khả năng trích xuất tốt các đặc trưng thị giác như màu sắc, hình dạng, cạnh và kết cấu. Các đặc trưng này phù hợp với bài toán phân loại chi tiết loài chim, trong đó sự khác biệt giữa các lớp thường nằm ở những chi tiết nhỏ trên ngoại hình.



Hình 5: Kiến trúc mạng VGG16

4.1.2 Trích xuất đặc trưng từ VGG16 pretrained trên ImageNet

Trong phương pháp này, nhóm sử dụng VGG16 đã được huấn luyện trước trên tập ImageNet làm bộ trích xuất đặc trưng [3]. Phần phân loại gốc của VGG16 được loại bỏ bằng cách đặt `include_top=False`. Toàn bộ backbone VGG16 được đóng

bằng, tức là các trọng số pretrained không được cập nhật trong quá trình huấn luyện.

Ảnh đầu vào có kích thước $224 \times 224 \times 3$ được đưa qua backbone VGG16 để tạo ra feature map có kích thước $7 \times 7 \times 512$. Sau đó, lớp `GlobalAveragePooling2D` được sử dụng để chuyển feature map thành một vector đặc trưng 512 chiều cho mỗi ảnh. Vector đặc trưng này được đưa vào Random Forest để phân loại bằng mô hình học máy truyền thống.

```
1 base_model = VGG16(  
2     include_top=False,  
3     weights="imagenet",  
4     input_shape=(224, 224, 3)  
5 )  
6  
7 base_model.trainable = False  
8  
9 inputs = layers.Input(shape=(224, 224, 3))  
10 x = base_model(inputs, training=False)  
11 features = layers.GlobalAveragePooling2D()(x)
```

Việc đóng băng backbone giúp giữ nguyên các đặc trưng thị giác đã học từ ImageNet, đồng thời giảm chi phí huấn luyện vì mô hình không cần cập nhật lại trọng số của VGG16. Trong phương pháp này, VGG16 chỉ đóng vai trò là bộ trích xuất đặc trưng, còn quá trình học phân loại được thực hiện bởi Random Forest. Sau bước trích xuất đặc trưng, mỗi ảnh được biểu diễn bởi một vector:

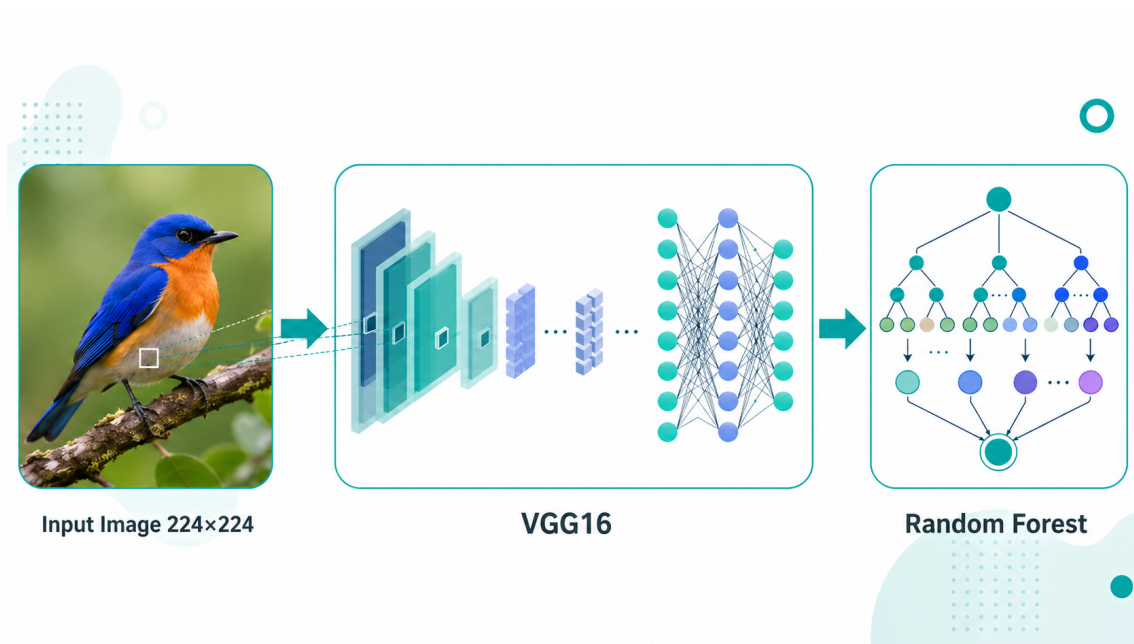
$$\mathbf{f}_i \in \mathbb{R}^{512}$$

Trong đó, $\mathbf{f}_i$ là vector đặc trưng của ảnh thứ i .

Bảng 2: Cấu hình bộ trích xuất đặc trưng VGG16

Thành phần	Chi tiết
Backbone	VGG16 pretrained trên ImageNet
Trạng thái trọng số	Frozen
Input size	$224 \times 224 \times 3$
Lớp pooling	GlobalAveragePooling2D
Chiều vector đặc trưng	512
Số mẫu train/val/test (ORIG)	20.080 / 4.248 / 4.468
Số mẫu train/val/test (AUG)	30.000 / 4.248 / 4.468
Thiết bị	Kaggle (GPU / CPU)

4.1.3 Phân loại bằng Random Forest



Hình 6: Quy trình phân loại bằng Random Forest

Nhóm sử dụng vector đặc trưng 512 chiều được trích xuất từ VGG16 làm đầu vào cho mô hình Random Forest [4]. Khác với lớp full connected, Random Forest không học trực tiếp từ ảnh gốc mà học từ ma trận đặc trưng có dạng:

$$X_{\text{train}} \in \mathbb{R}^{N \times 512}$$

Trong đó, mỗi hàng tương ứng với một ảnh và mỗi cột tương ứng với một đặc trưng được trích xuất từ VGG16.

```
1 feature_extractor = Model(inputs=inputs, outputs=features)
2
3 X_train_rf = feature_extractor.predict(train_ds)
4 X_test_rf = feature_extractor.predict(test_ds)
```

Sau khi thu được vector đặc trưng, nhóm sử dụng Grid Search để tìm bộ siêu tham số phù hợp cho Random Forest.[4]

Bảng 3: Không gian tìm kiếm siêu tham số Random Forest bằng Grid Search

Siêu tham số	Giá trị thử nghiệm	Ý nghĩa
n_estimators	{100, 200}	Số lượng cây quyết định trong Random Forest
max_depth	{10, 20}	Độ sâu tối đa của mỗi cây
min_samples_split	{2, 10}	Số mẫu tối thiểu để một nút được phép chia tiếp
min_samples_leaf	{1, 5}	Số mẫu tối thiểu tại mỗi nút lá
max_features	{sqrt}	Số đặc trưng được xét tại mỗi lần chia nhánh
bootstrap	{True}	Sử dụng lấy mẫu bootstrap khi xây dựng từng cây

Bảng 4: Bộ siêu tham số tốt nhất của Random Forest sau Grid Search

Siêu tham số	Giá trị tốt nhất
n_estimators	200
max_depth	20
min_samples_split	2
min_samples_leaf	5
max_features	sqrt
bootstrap	true
Best CV Accuracy	0.8536

4.2 Phương pháp Vision Transformer (ViT)

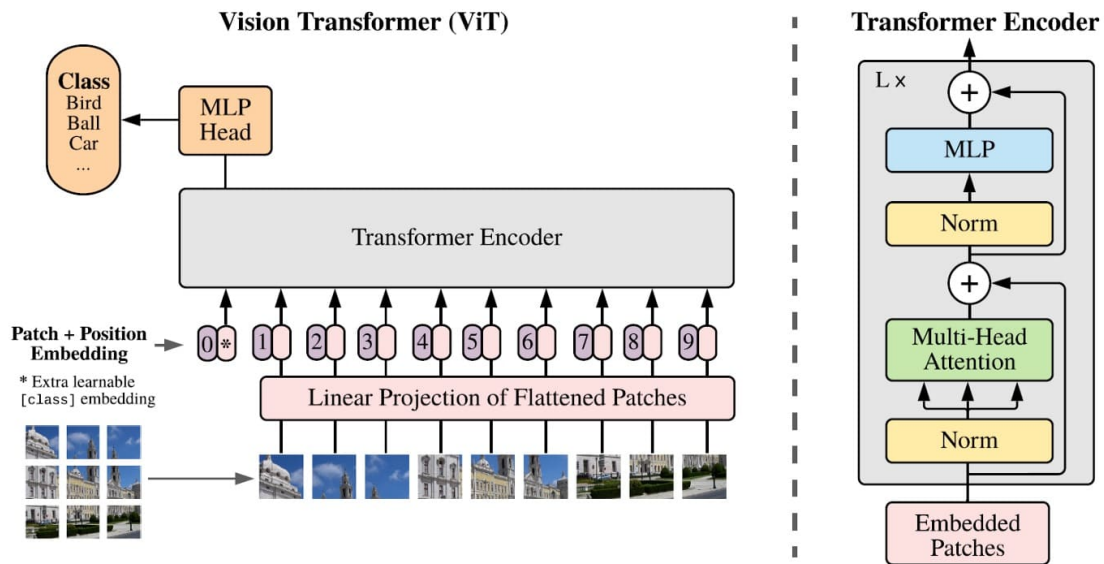
4.2.1 Tổng quan về ViT

Vision Transformer (ViT) là một mô hình học sâu được sử dụng cho xử lý hình ảnh. Nó sử dụng cơ chế Transformer, kiến trúc chủ yếu được sử dụng trong lĩnh vực xử lý ngôn ngữ tự nhiên, và áp dụng để có thể xử lý hình ảnh. Điều này cho phép ViT chuyển đổi hình ảnh thành các vector biểu diễn, từ đó mô hình có thể học và rút trích các đặc trưng quan trọng từ hình ảnh. [5]

4.2.2 Kiến trúc Mô hình ViT

Mô hình Vision Transformer gồm 3 thành phần chính:

1. **Linear Projection of Flattened Patches:** chia ảnh thành các patch, chuyển mỗi patch thành vector embedding làm đầu vào cho Transformer.
2. **Transformer Encoder:** dùng self-attention và MLP để học đặc trưng toàn cục và quan hệ giữa các vùng ảnh.
3. **Classification Head:** sử dụng biểu diễn từ CLS token để thực hiện phân loại ảnh



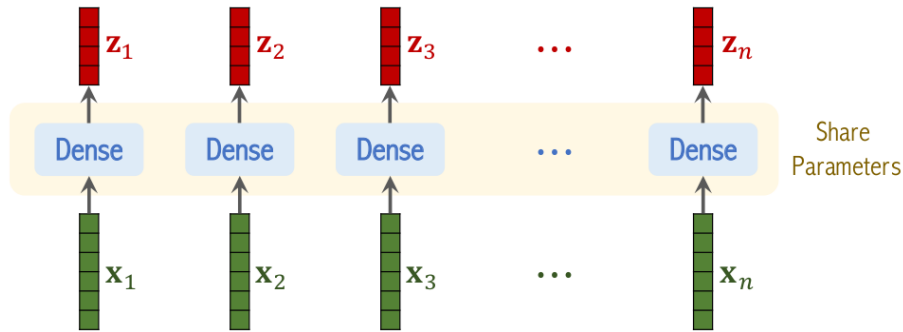
Hình 7: Kiến trúc của Vision Transformer

1. Linear Projection of Flattened Patches

- Patch Embedding

Với mỗi ảnh đầu vào, ViT xử lý bằng cách chia ảnh ra thành các phần có kích thước bằng nhau (patch)

Bước tiếp theo, đưa các patches này về dạng vector bằng cách flatten các patches này ra.



Hình 8: Minh họa quá trình Linear Projection của các Patch trong ViT

Hình trên mô tả phần Linear Projection. Thực chất Linear Projection là một lớp Dense với đầu vào là flatten vector của các patches, đầu ra sẽ là embedding vector tương ứng với từng patch.

$$z_i = Wx_i + b$$

Trong đó:

- x_i là flattened vector của patch thứ i .
- z_i là output tương ứng của x_i khi qua Linear Projection.
- W được gọi là ma trận embedding.

• Positional Embedding

Trong mô hình Vision Transformer, sau khi ảnh được chia thành các patch và chuyển thành các vector embedding, cần bổ sung Positional Embedding để cung cấp thông tin vị trí không gian của từng patch trong ảnh.

Do kiến trúc Transformer sử dụng cơ chế self-attention và xử lý đầu vào như một chuỗi token, mô hình không tự nhận biết thứ tự hay vị trí của các patch. Nếu chỉ dùng patch embedding, các patch có thể bị xem như hoán vị cho nhau mà không làm thay đổi đầu vào của mô hình, dẫn đến mất thông tin cấu trúc không gian của ảnh.

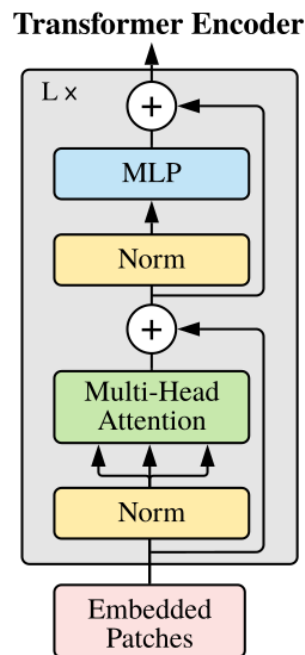
• Class Embedding

Bên cạnh các patch embedding, Vision Transformer bổ sung một Class Embedding (ký hiệu [CLS] token) vào đầu chuỗi đầu vào trước khi đưa vào Transformer Encoder.

Class token là một vector học được (learnable vector), có nhiệm vụ tổng hợp thông tin toàn cục của toàn bộ ảnh thông qua các lớp self-attention. Trong quá trình encoder xử lý, token này tương tác với tất cả các patch và dần chứa biểu diễn đặc trưng của toàn ảnh.

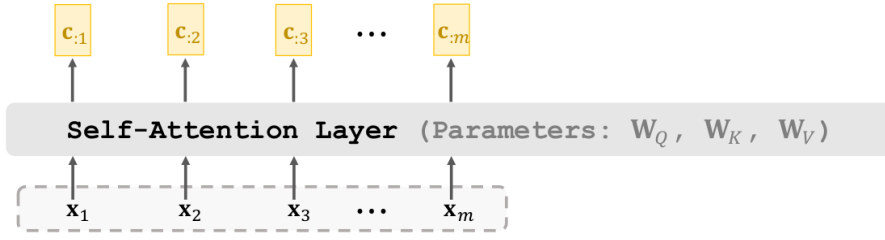
2. Transformer Encoder

Transformer Encoder là thành phần cốt lõi của Vision Transformer, gồm nhiều encoder block chứa Multi-Head Self-Attention và MLP. Thành phần này giúp mô hình học quan hệ toàn cục giữa các patch, trích xuất đặc trưng mức cao và tạo biểu diễn toàn ảnh cho bài toán phân loại.



Hình 9: Kiến trúc Transformer Encoder

- **Self Attention layer** Self attention layer là thành phần chính để tạo nên một block trong Transformer Encoder.[6]



Hình 10: Cấu trúc Self Attention

Đầu vào của Self attention layer là một chuỗi $X = [x_1, x_2, x_3, \dots, x_m]$

Đầu ra của Self Attention layer là một context vector C chứa những thông tin quan trọng nhất của chuỗi đầu vào $C = [c_1, c_2, c_3, \dots, c_m]$

Các parameters của layer này bao gồm W_Q, W_K, W_V .

Công thức Self-Attention

Self-Attention được tính theo công thức:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Trong đó:

- $Q = XW_Q$ là ma trận Query
- $K = XW_K$ là ma trận Key
- $V = XW_V$ là ma trận Value
- W_Q, W_K, W_V là các ma trận trọng số cần học
- d_k là chiều của vector Key (dùng để chuẩn hóa)

• Multi-head Attention

Multi-Head Attention mở rộng cơ chế Self-Attention bằng cách thực hiện nhiều phép attention song song, giúp mô hình học được các mối quan hệ khác nhau giữa các phần tử trong chuỗi đầu vào.

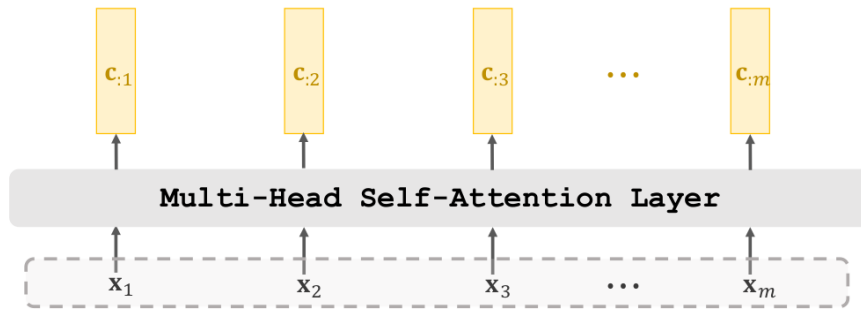
Công thức của Multi-Head Attention được định nghĩa như sau:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h)W^O$$

Trong đó, mỗi head được tính bằng:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

- h là số lượng head
- W_i^Q, W_i^K, W_i^V là các ma trận trọng số của head thứ i
- W^O là ma trận tuyến tính dùng để chiếu kết quả sau khi nối các head

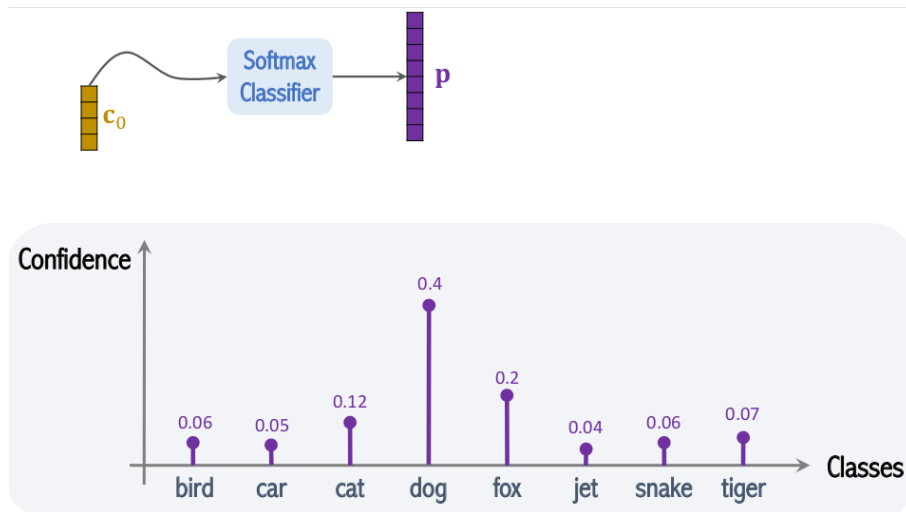


Hình 11: Cấu trúc Multi Head Attention

Ngoài thành phần chính là Multi-head Attention thì Transformer Encoder còn được tạo bởi các lớp khác như Add & Norm, Feed Forward, Add & Norm.

3. Classification Head

Classification Head là thành phần cuối của Vision Transformer, sử dụng biểu diễn của token [CLS] sau Transformer Encoder, sau đó đưa qua một hoặc nhiều lớp fully connected để dự đoán xác suất các lớp.



Hình 12: Classification Head

4.2.3 Mã nguồn ViT[1]

1. Khởi tạo mô hình ViT[7]

```

1 MODEL_NAME = "google/vit-base-patch16-224-in21k"
2 processor = AutoImageProcessor.from_pretrained(MODEL_NAME)
3 model = ViTModel.from_pretrained(MODEL_NAME)
4 model = model.to(device)
5 model.eval()

```

2. Định nghĩa hàm rút trích đặc trưng

```

1 def extract_features(dataloader, desc="Extracting"):
2     features = []
3     labels = []
4
5     with torch.no_grad():
6         for batch_images, batch_labels in tqdm(dataloader,
7             desc=desc):
8             batch_images = batch_images.to(device)
9
10            outputs = model(pixel_values=batch_images)
11            cls_features = outputs.last_hidden_state[:, 0, :]
12
13            features.append(cls_features.cpu().numpy())
14            labels.extend(batch_labels.numpy())
15
16    return np.vstack(features), np.array(labels)

```

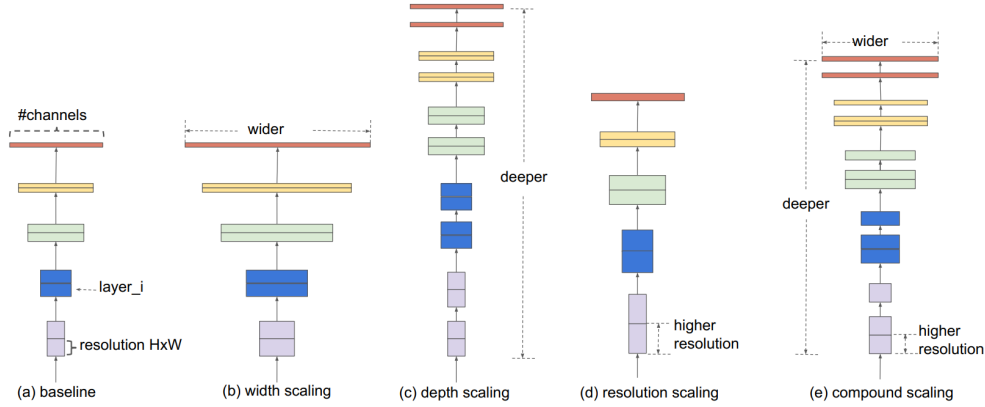
4.2.4 Lý do lựa chọn ViT-B/16 để rút trích đặc trưng

Kiến trúc ViT-B/16 được lựa chọn làm bộ trích xuất đặc trưng cho tập dữ liệu 150 loài chim dựa trên ba cơ sở chính. Thứ nhất, cơ chế Self-attention cho phép mô hình nắm bắt các phụ thuộc xa và đặc trưng toàn cục, đặc biệt hiệu quả trong bài toán phân loại hạt mịn (FGVC) vốn yêu cầu sự tổng hợp thông tin từ nhiều bộ phận phân tán trên cơ thể chim. Thứ hai, kích thước patch 16×16 giúp duy trì độ phân giải không gian ở mức chi tiết, bảo toàn các đặc trưng vi mô như họa tiết lông hay hình dạng mỏ để phân biệt các loài có hình thái tương đồng. Cuối cùng, cấu hình Base (86 triệu tham số) kết hợp cùng kỹ thuật học chuyển giao (Transfer Learning) từ ImageNet-21k tạo ra sự cân bằng tối ưu giữa khả năng biểu diễn đặc trưng và kiểm soát rủi ro quá khớp (overfitting), đảm bảo hiệu năng phân loại cao mà vẫn tiết kiệm tài nguyên tính toán.

4.3 Phương pháp 3: EfficientNetB0

4.3.1 Tổng quan về EfficientNetB0

- **Giới thiệu:** EfficientNet là họ mô hình được thiết kế nhằm đạt hiệu năng cao với chi phí tính toán thấp. Thay vì mở rộng mô hình theo từng chiều riêng lẻ như các kiến trúc truyền thống, EfficientNet áp dụng phương pháp *compound scaling* [8] để đồng thời cân chỉnh chiều sâu, chiều rộng và độ phân giải đầu vào theo một tỉ lệ hợp lý. EfficientNetB0 là biến thể cơ sở (baseline) của họ này, có số lượng tham số nhỏ nhất nhưng vẫn đạt hiệu năng cạnh tranh với nhiều kiến trúc lớn hơn.
- **Compound Scaling:** Để đạt được hiệu suất tối ưu, EfficientNet không chỉ mở rộng đơn lẻ một chiều mà sử dụng một hệ số tổng hợp ϕ để điều chỉnh đồng thời ba tham số quan trọng: chiều sâu (d), chiều rộng (w) và độ phân giải (r).



Hình 13: So sánh các phương pháp mở rộng mạng (Scaling) và cơ chế Compound Scaling của EfficientNet.

Việc điều chỉnh được thực hiện theo công thức:

$$\text{depth} : d = \alpha^\phi, \quad \text{width} : w = \beta^\phi, \quad \text{resolution} : r = \gamma^\phi$$

với ràng buộc $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$ và $\alpha, \beta, \gamma \geq 1$.

Bảng 5: Kiến trúc của EfficientNet B0

Stage	Operator	Resolution	#Channels	#Layers
i	$\hat{\mathcal{F}}_i$	$\hat{H}_i \times \hat{W}_i$	$\hat{C}_i$	$\hat{L}_i$
1	Conv3x3	224×224	32	1
2	MBConv1, k3x3	112×112	16	1
3	MBConv6, k3x3	112×112	24	2
4	MBConv6, k5x5	56×56	40	2
5	MBConv6, k3x3	28×28	80	3
6	MBConv6, k5x5	14×14	112	3
7	MBConv6, k5x5	14×14	192	4
8	MBConv6, k3x3	7×7	320	1
9	Conv1x1 & Pooling & FC	7×7	1280	1

- **Khối MBConv (Mobile Inverted Bottleneck Convolution):** Thành phần cốt lõi của EfficientNetB0 là các khối MBConv, kế thừa từ MobileNetV2[9]. Mỗi khối MBConv bao gồm:

- Lớp tích chập 1×1 để mở rộng số kênh (expansion).
- Lớp tích chập theo chiều sâu (depthwise convolution) với kernel 3×3 hoặc 5×5 .
- Cơ chế *Squeeze-and-Excitation* (SE)[10] giúp mô hình tái trọng số kênh theo độ quan trọng.
- Lớp tích chập 1×1 để thu nhỏ lại số kênh (projection).
- Kết nối tắt (skip connection) khi chiều đầu vào và đầu ra trùng nhau.

- **Lý do lựa chọn:** EfficientNetB0 được chọn vì khả năng trích xuất đặc trưng phong phú với số tham số nhỏ gọn (khoảng 5,3 triệu tham số), phù hợp với tài nguyên tính toán hạn chế. Đặc biệt, nhờ cơ chế Squeeze-and-Excitation, mô hình có khả năng tập trung vào các kênh đặc trưng quan trọng, hỗ trợ tốt cho bài toán phân loại chi tiết các loài chim có ngoại hình tương đồng.

4.3.2 Trích xuất đặc trưng

a) Xây dựng Feature Extractor

Backbone EfficientNetB0 pretrained trên ImageNet được tải với tùy chọn `include_top=False` để loại bỏ lớp phân loại gốc. Toàn bộ trọng số được đóng băng hoàn toàn (`trainable=False`), tức là không có bất kỳ quá trình fine-tune nào. Lớp `GlobalAveragePooling2D` được gắn thêm vào cuối để tổng hợp toàn bộ feature map thành một vector đặc trưng 1280 chiều cho mỗi ảnh đầu vào:

```
1 base = EfficientNetB0(include_top=False, weights='imagenet',
2                       input_shape=(224, 224, 3))
3 base.trainable = False
4 inputs = Input(shape=(224, 224, 3))
5 x = base(inputs, training=False)
6 x = GlobalAveragePooling2D()(x)
7 feature_extractor = Model(inputs, x)
```

b) Rút trích và lưu đặc trưng

Toàn bộ tập `trainval` và `test` được đưa qua feature extractor để thu được các vector đặc trưng thô. Các vector này được lưu dưới dạng file `.npy` để tái sử dụng mà không cần chạy lại mạng mỗi lần thực nghiệm, giúp tiết kiệm đáng kể thời gian tính toán:

```
1 def extract_features(model, generator):
2     generator.reset()
3     features = model.predict(generator, verbose=1)
4     labels = generator.classes
5     return features, labels
```

Sau khi rút trích, kích thước dữ liệu đặc trưng thu được như sau: tập **không tăng cường dữ liệu** có $N_{\text{train}} = 24328$ mẫu, tập **có tăng cường dữ liệu** có $N_{\text{train}} = 34248$ mẫu; tập test giữ cố định với $N_{\text{test}} = 4468$ mẫu, mỗi mẫu là vector $\mathbf{f} \in \mathbb{R}^{1280}$.

4.3.3 Cấu hình Feature Extractor

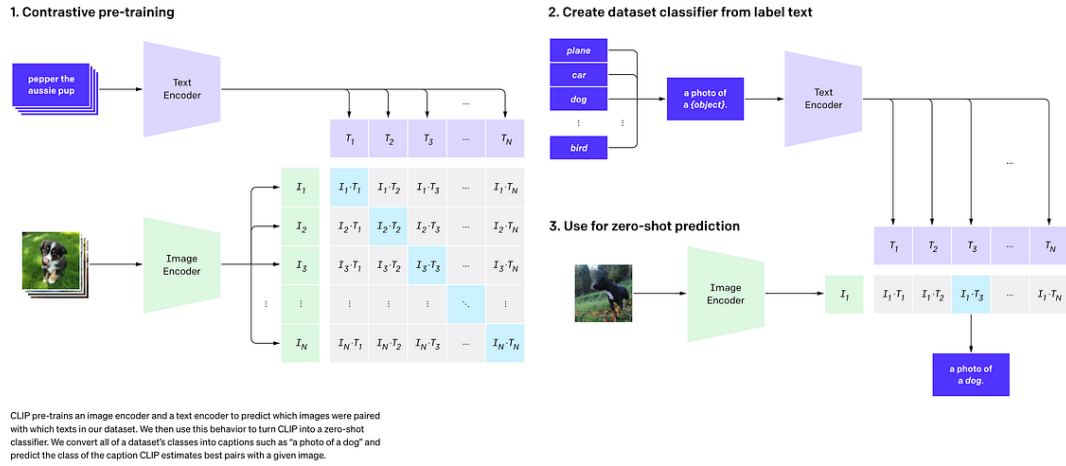
Bảng 6: Cấu hình bộ trích xuất đặc trưng EfficientNetB0

Thành phần	Chi tiết
Backbone	EfficientNetB0 pretrained trên ImageNet
Trạng thái trọng số	Frozen
Input size	$224 \times 224 \times 3$
Lớp pooling	GlobalAveragePooling2D
Chiều đặc trưng gốc	1280
Số mẫu train/val/test (ORIG)	20.080 / 4248 / 4468
Số mẫu train/val/test (AUG)	30.000 / 4248 / 4468
Thiết bị	Google Colab (GPU / CPU)

4.4 Phương pháp CLIP

4.4.1 Tổng quan về CLIP (Contrastive Language-Image Pre-training)

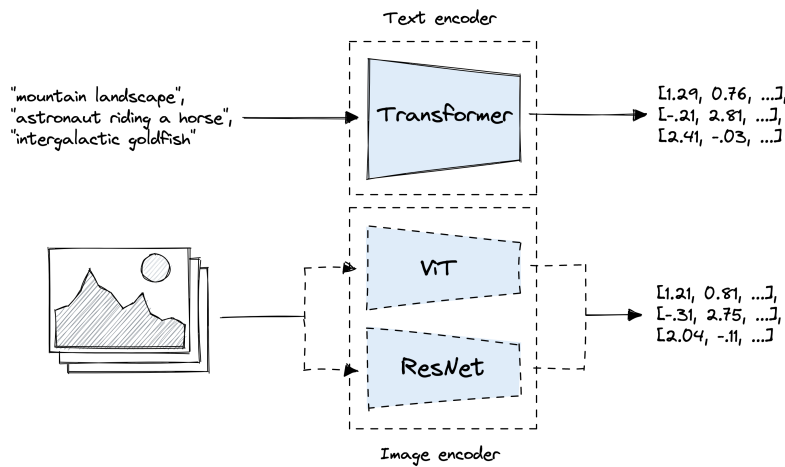
Giới thiệu: CLIP (Contrastive Language-Image Pre-training) là một mô hình học máy mang tính đột phá được giới thiệu bởi Alec Radford và cộng sự vào năm 2021, nhằm thu hẹp khoảng cách giữa xử lý dữ liệu hình ảnh và văn bản. Không giống như các mô hình truyền thống phụ thuộc vào các nhãn được định nghĩa sẵn, CLIP học cách liên kết hình ảnh với các mô tả bằng ngôn ngữ trên quy mô lớn. Nhờ đó, nó có thể thực hiện các tác vụ mà không cần dữ liệu huấn luyện chuyên biệt được gọi là phân loại zero-shot. [11]



Hình 14: Sơ đồ pipeline của mô hình CLIP

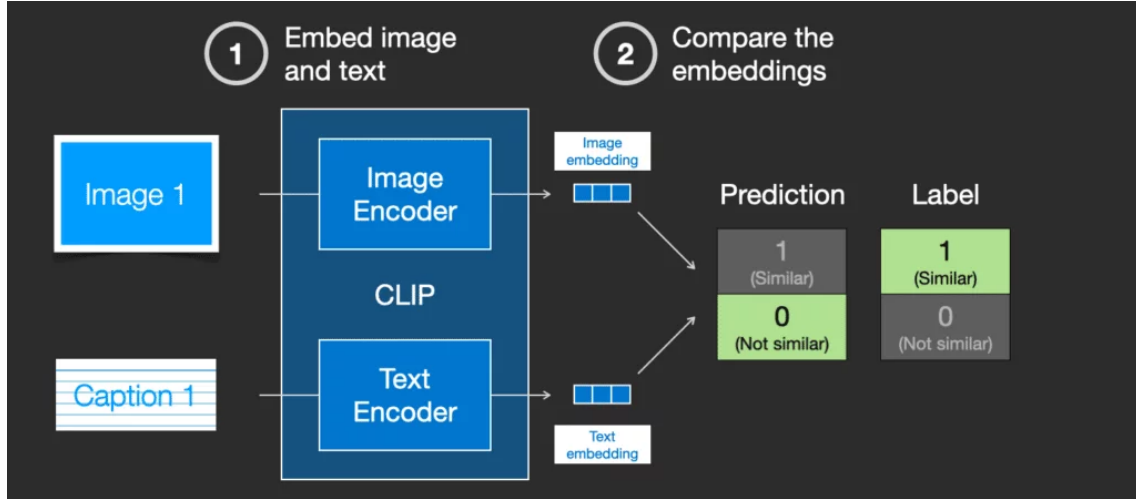
Dữ liệu huấn luyện: CLIP được huấn luyện trên tập dữ liệu gồm 400 triệu cặp (ảnh, văn bản) thu thập từ internet. Nhiệm vụ tiền huấn luyện đơn giản là dự đoán caption nào đi kèm với ảnh nào đây là cách hiệu quả và có khả năng mở rộng để học các biểu diễn ảnh chất lượng cao từ đầu. [11]

Kiến trúc mô hình: Kiến trúc của CLIP được xây dựng xung quanh hai bộ mã hóa riêng biệt: một bộ mã hóa thị giác để xử lý hình ảnh và một bộ mã hóa văn bản để phân tích các mô tả bằng lời. Bộ mã hóa thị giác sử dụng hai kiến trúc chính là ResNet và Vision Transformer (ViT). Đối với đầu vào văn bản, một Transformer 12 lớp được sử dụng với chiều rộng 512 và 8 attention head. Ở giai đoạn cuối của cả hai bộ mã hóa, một hàm global pooling được áp dụng để nén feature map thành một vector đặc trưng duy nhất. [12]



Hình 15: Kiến trúc của mô hình CLIP.

Contrastive Learning: Cốt lõi của CLIP là contrastive Learning — mô hình học bằng cách so sánh các cặp ảnh và văn bản. Mục tiêu huấn luyện là dự đoán cặp ảnh–văn bản nào là đúng trong số N cặp có thể có của một batch. Mô hình tối đa hóa độ tương đồng cosine giữa các embedding của cặp đúng, đồng thời tối thiểu hóa độ tương đồng cho các cặp sai. Mỗi batch xét toàn bộ N^2 cặp, với hàm mất mát cross-entropy đối xứng tối ưu hóa các điểm tương đồng cosine này. [13]



Hình 16: Cơ chế hoạt động của CLIP khi huấn luyện bằng contrastive learning.

Contrastive Loss (CLIP)

$$L = -\frac{1}{2n} \left(\sum_{i=1}^n \log \frac{\exp(\text{logits}_{i,i})}{\sum_{j=1}^n \exp(\text{logits}_{i,j})} + \sum_{j=1}^n \log \frac{\exp(\text{logits}_{j,j})}{\sum_{i=1}^n \exp(\text{logits}_{i,j})} \right)$$

$\text{logits}_{i,j}$ = temperature-scaled similarity score between i^{th} image and j^{th} text
 n = total number of positive pairs

*Note: logits matrix is generally **asymmetric***

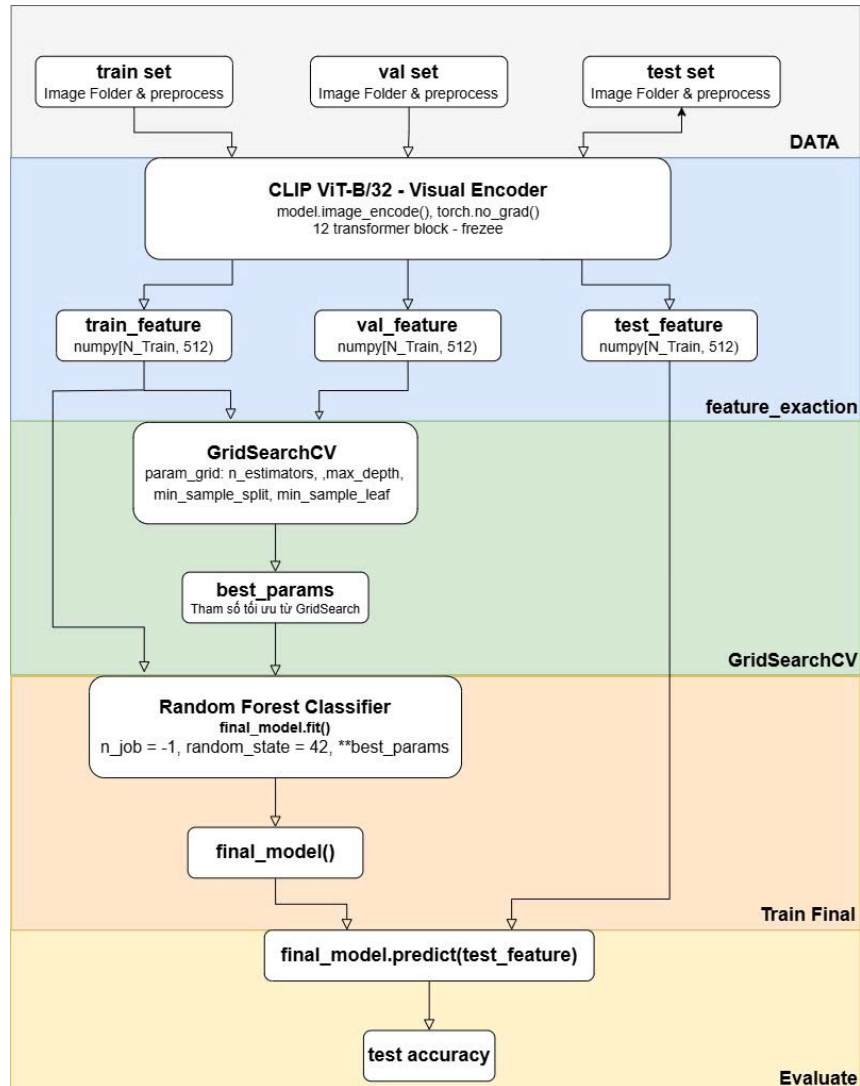
Hình 17: Công thức tính CLIP Contrastive Loss

Phạm vi đồ án: Nhóm chỉ sử dụng image encoder của CLIP để trích xuất đặc trưng từ ảnh mà không sử dụng text encoder. Lý do là bài toán đặt ra thuộc dạng phân loại có giám sát, trong đó dữ liệu đã có nhãn rõ ràng nên không cần đến thông tin ngôn ngữ. Image encoder của CLIP, với việc được huấn luyện trước trên tập dữ

liệu quy mô lớn, đã học được các đặc trưng thị giác có tính tổng quát cao, đủ để phục vụ hiệu quả cho bài toán. Việc không sử dụng text encoder giúp đơn giản hóa mô hình, giảm chi phí tính toán và dễ triển khai hơn, trong khi vẫn đảm bảo hiệu năng khi kết hợp với các bộ phân loại như SVM, Random Forest.

4.4.2 Huấn luyện mô hình

a) Sơ đồ các giai đoạn huấn luyện mô hình



Hình 18: Sơ đồ các giai đoạn huấn luyện mô hình

b) Hàm trích xuất đặc trưng từ model pretrain CLIP ViT-B/32

Trong bài toán phân loại, nhóm sử dụng CLIP để trích xuất đặc trưng ảnh, sau đó đưa vào các mô hình phân loại như SVM hoặc Random Forest. Toàn bộ tập dữ liệu huấn luyện được chuyển đổi thành vector số thông qua hàm

`get_features()`. Vector này sẽ mang nhiều thông tin, nắm bắt các đặc điểm hình ảnh quan trọng có khả năng phân biệt các lớp. Hàm này sử dụng bộ mã hóa hình ảnh của CLIP (ViT-B/32) đã được huấn luyện trước để rút trích đặc trưng mà không cập nhật trọng số.

Để đưa vào mô hình Random Forest, Dữ liệu cần có định dạng `n_samples`, `n_features`, trong đó mỗi hàng tương ứng với một ảnh và mỗi cột tương ứng với một đặc trưng. Do đó các embedding vector 512 chiều thu được từ bộ mã hóa CLIP ViT-B/32 được tích lũy rồi ghép thành chiều dọc thông qua `torch.cat()`, tạo thành ma trận có kích thước **(N, 512)**.

```
1 def get_features(dataset):
2     all_features = []
3     all_labels = []
4     with torch.no_grad():
5         for images, labels in tqdm(DataLoader(dataset,
6             batch_size=100)):
7             features = model.encode_image(images.to(device))
8             all_features.append(features)
9             all_labels.append(labels)
10    return torch.cat(all_features).cpu().numpy(),
11           torch.cat(all_labels).cpu().numpy()
```

Trong đó:

- `all_features`: ma trận đặc trưng kích thước $[N, 512]$, với N là tổng số ảnh trong tập dữ liệu, mỗi hàng là một vector đặc trưng 512 chiều của một ảnh.
- `all_labels`: vector kích thước $[N]$, chứa nhãn số nguyên tương ứng với từng ảnh.

Toàn bộ quá trình rút trích đặc trưng chỉ được thực hiện một lần duy nhất trước khi huấn luyện, sau đó các vector đặc trưng được giữ nguyên trong bộ nhớ.

5 Mô hình phân loại

Sau khi thực hiện rút trích đặc trưng trên toàn bộ dataset, do dữ liệu đã được chia sẵn thành tập train và validation, nhóm tiến hành gộp hai tập này lại nhằm phục vụ cho quá trình tìm kiếm siêu tham số tối ưu. Khi đó, đặc trưng và nhãn của hai tập được kết hợp thành một tập dữ liệu chung.

Để vẫn đảm bảo vai trò ban đầu của từng tập, nhóm sử dụng `PredefinedSplit`. Cụ thể, các mẫu thuộc tập train được gán giá trị -1 (chỉ dùng để huấn luyện), trong khi các mẫu thuộc tập validation được gán giá trị 0 (dùng để đánh giá). Cách tiếp cận này tương đương với việc sử dụng một tập validation cố định (hold-out validation), giúp tối ưu siêu tham số mà không làm thay đổi cấu trúc chia dữ liệu ban đầu.

```
1 X_train, y_train = train_features, train_labels
2 X_val, y_val = val_features, val_labels
3
4 X_combined = np.vstack((X_train, X_val))
5 y_combined = np.concatenate((y_train, y_val))
6
7 test_fold = np.concatenate([
8     np.full(X_train.shape[0], -1),
9     np.full(X_val.shape[0], 0)
10 ])
11
12 ps = PredefinedSplit(test_fold)
```

Trong mô hình Random Forest, nhóm sử dụng phương pháp `GridSearchCV` để tìm ra bộ siêu tham số tối ưu nhất cho bài toán phân loại loài chim. Tập tham số được lựa chọn dựa trên các đặc trưng phổ biến có ảnh hưởng lớn đến hiệu suất mô hình, bao gồm số lượng cây (`n_estimators`), độ sâu tối đa của cây (`max_depth`), số lượng mẫu tối thiểu để tách một node (`min_samples_split`), số mẫu tối thiểu tại lá (`min_samples_leaf`), và chiến lược chọn đặc trưng khi tách node (`max_features`), và phương pháp lấy mẫu (`bootstrap`).

```
1 param_grid = {
2     "n_estimators": [100, 200],
3     "max_depth": [10, 20],
4     "min_samples_split": [2, 10],
5     "min_samples_leaf": [1, 5],
6     "max_features": ["sqrt"],
7     "bootstrap": [True]
8 }
9
10 rf = RandomForestClassifier(n_jobs=-1, random_state=42)
11
12 grid = GridSearchCV(
13     estimator=rf,
14     param_grid=param_grid,
```

```

15     cv=ps,                # <--- i      cv=3 t h n h cv=ps
16     scoring="accuracy",
17     verbose=3,
18     n_jobs=-1
19 )
20 grid.fit(X_combined, y_combined)

```

Mô hình `RandomForestClassifier` được khởi tạo với `random_state = 42` nhằm đảm bảo tính tái lập. Quá trình tìm kiếm sử dụng chỉ số `accuracy`, phù hợp với bài toán phân loại đa lớp khi dữ liệu tương đối cân bằng. Đồng thời, tham số `n_jobs = -1` được thiết lập để tận dụng tối đa tài nguyên CPU, giúp tăng tốc quá trình huấn luyện.

Sau khi xác định được cấu hình tối ưu, mô hình Random Forest được huấn luyện lại trên toàn bộ tập huấn luyện với bộ tham số tốt nhất nhằm đạt hiệu suất dự đoán cao nhất trên tập kiểm tra

6 Thực nghiệm và kết quả

6.1 Đánh giá bộ dữ liệu

- **Tên bộ dữ liệu:** BIRDS 525 SPECIES- IMAGE CLASSIFICATION
- **Độ phân giải ảnh:** 224 x 224 với định dạng RGB.
- **Độ cân bằng dữ liệu:** Số lượng ảnh giữa các lớp tương đối đồng đều, dao động trong một khoảng hẹp (khoảng 120–180 ảnh mỗi lớp).
- **Độ đa dạng:** Ảnh đa dạng về các tư thế (bay, đậu, săn mồi) và đa dạng về nền của đối tượng như tán lá, cành cây, mặt nước, bầu trời giúp mô hình tăng khả năng phân biệt khỏi đối tượng khỏi nền.
- **Độ khó phân biệt:** Một số loài chim có đặc điểm hình thái tương đồng như màu sắc và hình dáng, khiến mô hình dễ nhầm lẫn giữa các lớp.
- **Lý do chọn:** Nhóm quyết định lựa chọn tập dữ liệu 'BIRDS 525 SPECIES' vì giữa các loài chim đôi khi chỉ khác biệt một chút về màu sắc, mỏ và chân, đồng thời kích thước được chuẩn hóa 224*224*3 phù hợp với bài toán phân loại chi tiết. Bên cạnh đó nhóm quyết định chỉ huấn luyện và thực nghiệm trên 150 loài có số lượng ảnh lớn nhất để giảm sự chênh lệch lớn của tập dữ liệu gốc đồng thời đảm bảo được thời gian và tài nguyên huấn luyện.

- **Nguồn Dataset:** BIRDS 525 SPECIES- IMAGE CLASSIFICATION

6.2 Thực nghiệm

Thực hiện đánh giá mô hình trên các độ đo đánh giá sau: Accuracy, Precision, Recall và Macro F1.

- **Accuracy** đánh giá hiệu năng tổng thể của mô hình trên toàn bộ tập dữ liệu.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision** phản ánh mức độ chính xác của các dự đoán, đặc biệt quan trọng khi các loài chim có đặc điểm tương đồng dễ gây nhầm lẫn.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall** đo khả năng mô hình nhận diện đầy đủ các mẫu của từng lớp, giúp phát hiện các lớp khó hoặc dễ bị bỏ sót.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **Macro F1** là trung bình điều hòa giữa Precision và Recall trên tất cả các lớp, cung cấp đánh giá cân bằng và toàn diện về chất lượng mô hình.

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Bên cạnh các độ đo về độ chính xác, nhóm còn xem xét các tiêu chí liên quan đến hiệu năng và khả năng triển khai thực tế của mô hình:

- **Feature Extraction Time:** Thời gian trung bình cần thiết để mã hóa (encode) một ảnh đầu vào thành vector đặc trưng thông qua mô hình CLIP.
- **Feature dim:** Kích thước của vector đặc trưng đầu ra sau quá trình trích xuất đặc trưng
- **Training RF Time:** Thời gian huấn luyện mô hình Random Forest trên bộ tham số tối ưu tìm được thông qua GridSearch.
- **Inference Time (CPU):** Thời gian mô hình dự đoán trung bình trên 1 ảnh.

- **Model size:** Kích thước mô hình, phản ánh thông qua số lượng tham số và dung lượng bộ nhớ cần thiết.

6.3 Kết quả thực nghiệm

Kết quả của quá trình thực nghiệm của các mô hình (mô hình sử dụng biến đổi dữ liệu) được thể hiện trong bảng dưới đây:

Bảng 7: Bảng so sánh kết quả sử dụng tài nguyên trên bốn phương pháp giải quyết

Tiêu chí	Feature dim	Extraction time (CPU/GPU)	Train RF time	Inference CPU	Model size
Phương pháp VGG16	512	198.6/5.3ms	3.53m	7.589ms	1239 MB
Phương pháp ViT-B/16	768	208.9/15.3ms	4.5m	40ms	218 MB
Phương pháp EfficientNetB0	1280	70/11 ms	5.8m	55ms	976 MB
Phương pháp CLIP	512	94.7/8.0ms	2.62m	46.8ms	1096.8 MB

Kết quả của quá trình huấn luyện mô hình trong 2 trường hợp có và không tăng cường dữ liệu dữ liệu đầu vào được thể hiện trong bảng dưới đây:

Bảng 8: So sánh kết quả với và không sử dụng Data Augmentation

Tiêu chí	Không Augmentation				Có Augmentation			
	Acc	F1	Recall	Prec	Acc	F1	Recall	Prec
Phương pháp VGG16	0.8487	0.8479	0.8482	0.8598	0.8579	0.8572	0.8584	0.8671
Phương pháp ViT-B/16	0.9570	0.9569	0.9570	0.9590	0.9597	0.9599	0.9601	0.9613
Phương pháp EfficientNetB0	0.9147	0.9148	0.9155	0.9187	0.9205	0.9204	0.9216	0.9238
Phương pháp CLIP	0.9038	0.9040	0.9036	0.9096	0.9078	0.9081	0.9085	0.9126

Các ví dụ phân loại đúng của phương pháp Vision Transformer(ViT)



Hình 19: Các ví dụ phân loại đúng của ViT

Các ví dụ phân loại sai của phương pháp Vision Transformer(ViT)



Hình 21: Phân loại sai của phương pháp ViT

Hình 20: Phân loại sai của phương pháp ViT

Cột bên trái là hình ảnh phân loại của loài chim bị gán label sai, có kèm theo label đúng của loài chim đó. Cột bên phải là hình ảnh đúng của loài mà model đoán sai

6.4 Nhận xét kết quả

6.4.1 So sánh tổng thể 4 phương pháp

- Về hiệu năng phân loại:** Phương pháp ViT-B/16 kết hợp Random Forest mang lại hiệu quả vượt trội nhất với độ chính xác (Accuracy) và F1-score đạt xấp xỉ 96% ở cả hai kịch bản dữ liệu. Điều này minh chứng cho sức mạnh của cơ chế Self-attention trong việc trích xuất tốt các đặc trưng toàn cục lẫn các chi tiết vi mô trên những loài chim có hình thái tương đồng. EfficientNetB0 và CLIP cũng cho kết quả rất khả quan (đạt trên 90%), trong khi VGG16 thể hiện kém nhất với độ chính xác chỉ khoảng 85.7%.

- **Về tài nguyên và tốc độ:** ViT-B/16 thể hiện sự tối ưu tuyệt vời về mặt không gian lưu trữ khi có kích thước mô hình nhỏ nhất (chỉ 218 MB), nhẹ hơn rất nhiều so với các kiến trúc đồ sộ như VGG16 (1239 MB) hay CLIP (1096.8 MB). Tuy nhiên, VGG16 lại có tốc độ dự đoán (Inference Time) ở mô hình học máy cực kỳ nhanh, chỉ mất 7.589ms. Về phần EfficientNetB0, mạng này có tốc độ trích xuất đặc trưng trên CPU nhanh nhất (70ms) nhưng lại tốn nhiều thời gian để huấn luyện Random Forest nhất (5.8 phút) do mô hình học máy phải xử lý vector đặc trưng có số chiều lớn nhất (1280 chiều).
- **Kết luận phương pháp:** ViT-B/16 là mô hình tối ưu và toàn diện nhất cho bài toán này nhờ sự cân bằng xuất sắc giữa độ chính xác cực cao và dung lượng mô hình nhỏ gọn, dễ dàng triển khai thực tế.

6.4.2 Ảnh hưởng của tăng cường dữ liệu (Data Augmentation)

- **Cải thiện độ chính xác tổng thể:** Việc áp dụng các kỹ thuật biến đổi ảnh (như xoay, lật, điều chỉnh độ sáng) đã giúp gia tăng nhẹ độ chính xác trên toàn bộ cả 4 mô hình thực nghiệm. Sự nâng cấp này thể hiện rõ nét nhất ở phương pháp EfficientNetB0 khi Accuracy tăng từ 91.47% lên mức 92.05%.
- **Giải quyết mất cân bằng và tăng tính vững (Robustness):** Bằng cách gia tăng và cân bằng số lượng mẫu lên mức 200 ảnh cho tất cả các lớp, kỹ thuật Data Augmentation giúp mô hình học được các biểu diễn đặc trưng bất biến trước sự đa dạng của môi trường, ánh sáng và góc chụp, từ đó nâng cao tính tổng quát của hệ thống.
- **Đánh giá biên độ tác động:** Cả 4 bộ trích xuất đặc trưng được nhóm sử dụng đều đã được học chuyển giao (Transfer Learning) từ các tập dữ liệu khổng lồ (như ImageNet hay tập 400 triệu cặp ảnh-văn bản của CLIP), do đó chúng vốn đã sở hữu sẵn khả năng tổng quát hóa rất mạnh. Vì lý do này, việc gia tăng thêm dữ liệu bằng Augmentation trên tập huấn luyện hiện tại mang lại giá trị tích cực nhưng biên độ tăng trưởng không quá lớn (chỉ cải thiện dưới 1% độ chính xác cho hầu hết các phương pháp).

7 Đánh giá, kết luận và mở rộng:

7.1 Đánh giá

7.1.1 Ưu điểm

- Trong đề án này, nhóm đã xây dựng và đánh giá hệ thống phân loại chi tiết loài chim trên tập dữ liệu Birds-525. Một trong những ưu điểm nổi bật của đề án là việc tiếp cận bài toán theo nhiều hướng khác nhau ở giai đoạn trích xuất đặc trưng. Cụ thể, nhóm đã triển khai và so sánh các mô hình học sâu đã được huấn luyện trước như VGG16, ViT-B/16, EfficientNetB0 và CLIP. Cách tiếp cận này giúp làm rõ sự khác biệt về khả năng biểu diễn đặc trưng giữa các kiến trúc mạng khác nhau trong bài toán phân loại chi tiết.
- Kết quả thực nghiệm cho thấy các mô hình pretrained mang lại hiệu quả tốt khi được sử dụng làm bộ trích xuất đặc trưng. Đặc biệt, phương pháp ViT-B/16 kết hợp với Random Forest đạt hiệu năng cao nhất, với độ chính xác xấp xỉ 96% trên tập kiểm tra. Điều này cho thấy cơ chế Self-Attention của Vision Transformer có khả năng khai thác tốt các đặc trưng toàn cục và các chi tiết nhỏ trên ảnh chim, vốn là yếu tố quan trọng trong bài toán fine-grained classification.
- Việc kết hợp các mô hình học sâu làm bộ trích xuất đặc trưng với bộ phân loại truyền thống như Random Forest cũng cho thấy tính linh hoạt trong thiết kế hệ thống. Thay vì huấn luyện toàn bộ mạng nơ-ron từ đầu, nhóm chỉ sử dụng backbone pretrained để mã hóa ảnh thành vector đặc trưng, sau đó huấn luyện Random Forest trên các vector này. Cách làm này giúp giảm chi phí huấn luyện, dễ triển khai, dễ so sánh giữa các phương pháp và phù hợp với điều kiện tài nguyên tính toán còn hạn chế.
- Bên cạnh đó, nhóm cũng áp dụng kỹ thuật tăng cường dữ liệu nhằm cân bằng số lượng mẫu giữa các lớp và cải thiện khả năng tổng quát hóa của mô hình. Kết quả cho thấy việc sử dụng Data Augmentation giúp cải thiện nhẹ các độ đo đánh giá trên hầu hết các phương pháp, chứng minh vai trò tích cực của việc mở rộng dữ liệu huấn luyện trong bài toán phân loại ảnh nhiều lớp.

7.1.2 Hạn chế

- Mặc dù đạt được những kết quả tích cực, hệ thống vẫn còn tồn tại một số hạn chế. Trước hết, bài toán phân loại loài chim thuộc nhóm fine-grained classification nên nhiều lớp có đặc điểm hình thái rất tương đồng. Một số loài chim chỉ khác nhau ở các chi tiết nhỏ như màu lông, hoa văn, hình dạng mỏ, dáng đứng hoặc kích thước cơ thể. Khi ảnh đầu vào không thể hiện rõ các chi tiết này, mô hình vẫn có khả năng đưa ra dự đoán sai.
- Bên cạnh đó, trong phạm vi đề án, các mô hình pretrained chủ yếu được sử dụng ở chế độ đóng băng trọng số và chỉ đóng vai trò là bộ trích xuất đặc trưng. Do đó, hệ thống chưa thực hiện fine-tuning trực tiếp trên tập dữ liệu chim. Điều này giúp giảm thời gian huấn luyện nhưng cũng làm hạn chế khả năng thích nghi của mô hình với các đặc trưng chuyên biệt của bộ dữ liệu Birds-525.
- Ngoài ra, Random Forest không học trực tiếp từ ảnh gốc mà phụ thuộc hoàn toàn vào chất lượng vector đặc trưng được trích xuất từ backbone. Nếu vector đặc trưng chưa biểu diễn đủ tốt sự khác biệt giữa các loài có ngoại hình gần giống nhau, bộ phân loại phía sau cũng khó cải thiện đáng kể kết quả. Một số mô hình như VGG16 và CLIP còn có kích thước khá lớn, gây tốn dung lượng lưu trữ và có thể chưa phù hợp nếu triển khai trên thiết bị có tài nguyên hạn chế.
- Cuối cùng, hệ thống chưa khai thác sâu các kỹ thuật chuyên biệt cho bài toán fine-grained classification, chẳng hạn như cơ chế chú ý vào vùng đặc trưng của đối tượng, phát hiện bộ phận quan trọng của chim hoặc các phương pháp học biểu diễn phân cấp. Mặc dù Data Augmentation giúp cải thiện khả năng tổng quát của mô hình trước các thay đổi về góc chụp, ánh sáng và tư thế, hệ thống vẫn còn gặp khó khăn trong những trường hợp ảnh có nền phức tạp, đối tượng nhỏ, bị che khuất hoặc không thể hiện rõ các đặc điểm phân biệt giữa các loài.

7.2 Kết luận

- Trong đề án này, nhóm đã xây dựng hệ thống phân loại chi tiết loài chim trên tập dữ liệu Birds-525 với phạm vi thực nghiệm gồm 150 loài. Hướng tiếp cận chính là sử dụng các mô hình pretrained để rút trích đặc trưng ảnh, sau đó đưa vector đặc trưng vào Random Forest để thực hiện phân loại.

- Kết quả thực nghiệm cho thấy các mô hình học chuyển giao có khả năng hỗ trợ tốt cho bài toán phân loại ảnh nhiều lớp. Trong các phương pháp đã thử nghiệm, ViT-B/16 kết hợp Random Forest đạt kết quả tốt nhất về Accuracy, Precision, Recall và F1-score. EfficientNetB0 và CLIP cũng cho kết quả khả quan, trong khi VGG16 tuy có hiệu năng thấp hơn nhưng vẫn có ưu điểm là kiến trúc đơn giản, dễ giải thích và tốc độ dự đoán nhanh.
- Nhìn chung, hệ thống đã đáp ứng được mục tiêu đề ra là xây dựng, thực nghiệm và so sánh nhiều phương pháp rút trích đặc trưng khác nhau cho bài toán phân loại loài chim. Việc kết hợp giữa mô hình pretrained và Random Forest giúp giảm chi phí huấn luyện, tận dụng tốt các đặc trưng đã học từ tập dữ liệu lớn và tạo nền tảng cho các hướng cải tiến tiếp theo.

7.3 Mở rộng

- Trong tương lai, hệ thống có thể được mở rộng theo nhiều hướng khác nhau. Trước hết, nhóm có thể tiến hành fine-tuning một số lớp cuối của các backbone pretrained trên tập dữ liệu chim để giúp mô hình học được các đặc trưng chuyên biệt hơn cho bài toán. Đây là hướng có tiềm năng cải thiện hiệu năng, đặc biệt với các loài chim có ngoại hình tương đồng.
- Bên cạnh đó, nhóm có thể thử nghiệm thêm các bộ phân loại khác như SVM, XGBoost, LightGBM hoặc mạng neural network tùy chỉnh để so sánh với Random Forest. Việc mở rộng lên toàn bộ 525 loài trong tập Birds-525 cũng là một hướng phát triển quan trọng nhằm đánh giá khả năng mở rộng của hệ thống trên quy mô dữ liệu lớn hơn.
- Ngoài ra, hệ thống có thể được tối ưu để triển khai thực tế thông qua các kỹ thuật như giảm kích thước mô hình, nén mô hình hoặc chuyển đổi sang các định dạng nhẹ hơn. Một hướng mở rộng khác là xây dựng giao diện cho phép người dùng tải ảnh chim lên và nhận kết quả dự đoán kèm độ tin cậy, từ đó tăng tính ứng dụng của hệ thống trong thực tế.

TÀI LIỆU THAM KHẢO

- [1] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, “Transformers: State-of-the-art natural language processing,” pp. 38–45, 2020.
- [2] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” in *International Conference on Learning Representations*, 2015.
- [3] Keras, “Vgg16 and vgg19,” <https://keras.io/api/applications/vgg/>, 2024, tài liệu mô hình VGG16 và VGG19 trong Keras Applications.
- [4] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [5] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly *et al.*, “An image is worth 16x16 words: Transformers for image recognition at scale,” *arXiv preprint arXiv:2010.11929*, 2020.
- [6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [7] H. Face, “Vision transformer (vit) documentation,” 2023, tài liệu hướng dẫn sử dụng mô hình ViT-B/16 thông qua thư viện Transformers. [Online]. Available: https://huggingface.co/docs/transformers/model_doc/vit
- [8] M. Tan and Q. V. Le, “Efficientnet: Rethinking model scaling for convolutional neural networks,” in *Proceedings of the 36th International Conference on Machine Learning*, 2019, pp. 6105–6114.
- [9] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “Mobilenetv2: Inverted residuals and linear bottlenecks,” in *CVPR*, 2018, pp. 4510–4520.
- [10] J. Hu, L. Shen, and G. Sun, “Squeeze-and-excitation networks,” in *CVPR*, 2018, pp. 7132–7141.
- [11] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, “Learning transferable visual models from natural language supervision,” in *Proceedings*

of the 38th International Conference on Machine Learning, ser. Proceedings of Machine Learning Research, vol. 139, 2021, pp. 8748–8763.

- [12] X. Pan, T. Ye, D. Han, S. Song, and G. Huang, “Contrastive language-image pre-training with knowledge graphs,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [13] Eleventh Hour Enthusiast, “Learning transferable visual models from natural language supervision: Paper review,” <https://medium.com/@EleventhHourEnthusiast/learning-transferable-visual-models-from-natural-language-supervision-9d4cc9bc9af2>, 2024, accessed: 21-Apr-2026.